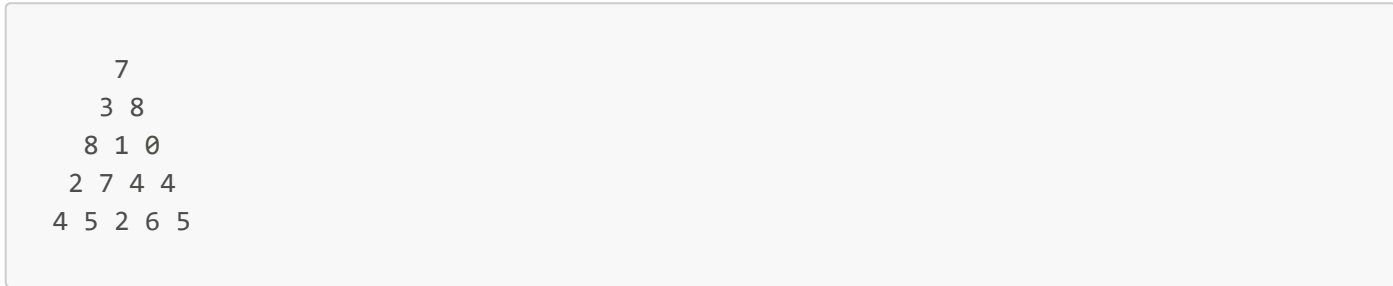


数字三角形问题

一、问题描述

有n (n≤100) 行数字，第i行有i个数字 (数字非负)，这些数字组成一个三角形。初始位置在三角形的最上方数字，每一步可以走到左下方或者右下方的数字。求从最上方到最下方的路径中，数字之和的最大值。

示例：



最优路径：7 → 3 → 8 → 7 → 5 = 30

二、问题分析

2.1 问题特点

- 1. 最优子结构：到达某个位置的最优路径，必然包含到达其上方相邻位置的最优路径；
- 2. 重叠子问题：不同路径会经过相同的中间节点，存在重复计算；
- 3. 无后效性：当前位置的最优解只与下方的子问题有关，与如何到达当前位置无关。

这些特点表明该问题适合使用动态规划求解。

三、动态规划解法

3.1 状态定义

定义 $dp[i][j]$ 表示：从位置 (i, j) 出发到达底部的最大路径和。

其中 i 表示行号， j 表示列号 (都从1开始)。

3.2 状态转移方程

对于位置 (i, j) ，可以向左下 $(i+1, j)$ 或右下 $(i+1, j+1)$ 移动。

因此，从 (i, j) 到底部的最大路径和等于当前位置的值加上两个方向中较大的路径和：

$$dp[i][j] = triangle[i][j] + \max(dp[i+1][j], dp[i+1][j+1])$$

3.3 边界条件

最后一行无法继续向下移动，因此：

$$dp[n][j] = triangle[n][j], \quad j = 1, 2, \dots, n$$

3.4 计算顺序

采用自底向上的计算顺序：

1. 首先初始化最后一行的 dp 值
2. 从第 $n-1$ 行开始，逐行向上计算
3. 最终 $dp[1][1]$ 即为答案

四、代码实现

```
#include <iostream>
#include <algorithm>
using namespace std;

int main(){
    int n;
    cin >> n;

    int triangle[105][105];
    int dp[105][105];

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cin >> triangle[i][j];
        }
    }

    for (int j = 1; j <= n; j++) {
        dp[n][j] = triangle[n][j];
    }

    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = triangle[i][j] + max(dp[i + 1][j], dp[i + 1][j + 1]);
        }
    }

    cout << dp[1][1] << endl;

    return 0;
}
```

五、复杂度分析

5.1 时间复杂度

- 需要遍历三角形中的每个元素一次
- 三角形共有 $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$ 个元素
- 时间复杂度： $O(n^2)$

5.2 空间复杂度

- 使用二维数组存储原始数据和dp状态
- 空间复杂度： $O(n^2)$

六、总结

要素	内容
状态定义	<code>dp[i][j]</code> 表示从位置 (i,j) 到底部的最大路径和
转移方程	<code>dp[i][j] = triangle[i][j] + max(dp[i+1][j], dp[i+1][j+1])</code>
边界条件	<code>dp[n][j] = triangle[n][j]</code>
计算顺序	自底向上 (从第 <code>n</code> 行到第 <code>1</code> 行)
最终答案	<code>dp[1][1]</code>